Document 19 — MLOps & LLMOps (Running What You Built)

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

Added at the end on purpose: MLOps only makes sense once there's a real system to run — you need Project 5 (Docs 12-18) working before "how do I safely change it in production" is a real question, instead of just a theoretical one.

Prerequisites

18_capstone (Project 5 built and working)

How to Read & Practice This Document

- **What:** the discipline of safely changing, deploying, and watching an AI system *after* it's already live — versions, gated releases, rolling back, catching slow decline.
- **Why:** Doc12 taught you how to ship a system once. This document is about what happens on day 2 — when you want to change a prompt, swap a model, or fix an agent, without breaking what's already working for real users.
- **When:** the moment your system has real users (or you're treating it as if it will) — not important for a one-off script, essential for anything you'd call "production."
- **How to practice:** 1. Read the Core Concepts below — this document's outside reading is kept small on purpose, since MLOps for LLM systems is a newer, faster-changing area than the rest of this curriculum, without as much settled reference material. 2. Do the **Basic/Intermediate** exercises against your own Project 5. 3. Do the **Real-world/Failure** exercises for real — actually trigger a rollback, actually act out a slow-decline situation. 4. Use **Hint 1** → **Hint 2** only after really trying. Ask for the full solution only if you say **"Show me the solution."** 5. Before calling this "done," explain out loud what would need to be true for you to trust an automatic rollback enough to not watch every release yourself. If you can't, you're not done.

The Story — what this document is actually building

Picture Project 5 already live, with real people depending on it, and you want to improve one of its prompts. Everything in this document is one connected story about how to make that change safely

— the same discipline a classic ML team already applies to a newly retrained model, just aimed at prompts and settings instead, since here nothing gets retrained.

- **First, the prompt has to become a real, tracked thing.** Not text you edit in place and hope you remember what changed — a named, saved version, so "which version was live when this happened" is always an answerable question.
- **Second, a new version has to earn its way to production.** Doc13 already built a test suite that can catch a regression when you run it by hand — the release gate is what makes running it *automatic and mandatory*, not optional discipline that's easy to skip under time pressure. No passing score, no release.
- **Third, even a version that passed the gate doesn't go straight to everyone.** A canary release hands it a small slice of real traffic first, or a shadow release runs it silently alongside the old version with nobody seeing its answers, so a problem the test suite missed shows up in real numbers before it reaches your whole user base.
- **Fourth, "healthy at launch" isn't the end of the story.** Costs creep up, the questions users ask drift, the documents behind a RAG system go stale — a slow decline like that never throws an error, so the same kind of score that gated the release has to keep being tracked over time, watching for a trend, not just a single bad request.
- **Fifth, the moment the gate or the ongoing watching shows a version is actually worse, rollback is the safety net.** It snaps the "live" pointer back to the last version *known to have passed* — automatically, not "whatever came before, good or bad," and not waiting on a person to notice a dashboard.

All five pieces answer one question: how do you keep changing an AI system without ever betting your whole user base on an untested guess? That's exactly what this document's Build Task turns into one real, working gate for Project 5. It's also why proving the gate can't be quietly bypassed, and having a backup plan for when your one outside provider has an outage, both matter just as much — a gate someone can walk around, or a system with no plan B for its only dependency, isn't actually safety. It just looks like it, right up until the day it isn't.

Jump to: Core Concepts · Practice Exercises · Build Task

Core Concepts (read this first — everything you need is here)

MLOps vs. LLMOps: what's actually different here

MLOps ("machine learning operations") is the set of habits a team uses to change, release, and watch a machine learning model safely in production. **LLMOps** is the same set of habits for systems built on large language models (LLMs) like GPT. Classic MLOps is built around a training loop: collect new data, retrain the model, check it, ship it, and watch for the data changing so much that you must retrain again. Most LLM systems skip that loop completely. You do not train the model — a provider like OpenAI does. What you change is everything *around* the model: prompts, the model

name, `temperature` (the setting that controls how random the answers are), tool settings, search sources for RAG, and the code that connects the agents.

Why this difference matters: LLMOps borrows MLOps's discipline (versions, gated releases, watching, rolling back), but points it at a different set of things. In your system, a one-line prompt change is your "new model." It can change behavior for every user, so it deserves the same care a classic ML team gives a newly retrained model — even though nothing was trained. Without this view, people treat a prompt edit as "just text," skip testing, and break production.

How it works — what changes, in each world:

Thing	Classic MLOps	LLMOps (your systems)	Small example
What you release	New model weights (the trained numbers inside the model) after retraining	A new prompt, model name, setting, or tool config	`writer_prompt` v3 → v4
What makes it worse over time	Data drift: new data no longer looks like the training data	Input drift, stale RAG documents, provider model updates, cost creep	Users start asking about a product launched last week
How you test a release	Accuracy on a held-out test set (data kept aside, never used for training)	Doc13's eval suite: test cases + scores (often an LLM judge)	`pass_rate >= 0.90`
Cost of one change	Hours or days of training on GPUs	Seconds to edit — which is exactly why it is easy to skip the checks	Edit a string, redeploy
Main outside risk	Your own training pipeline breaks	A provider outage, a rate limit, a model being retired	OpenAI returns HTTP 503

When to use it / when NOT to:

Situation	What to do	Why	Small example
A one-off script you run once and delete	Skip LLMOps	Nobody depends on it; the process costs more than the risk	A script to rename 200 files
A system with real users (chatbot, support desk)	Use the full LLMOps loop	A bad prompt change hurts real people at once	Project 5 serving content requests
You fine-tune your own model	Use classic MLOps for the model, plus LLMOps for prompts around it	Now you have both a training loop and prompts	A fine-tuned support classifier plus a reply prompt
A scheduled job that writes reports every	Use versions + a release gate; canary is less useful	There is no live traffic to split, but a bad prompt still	Nightly sales summary job

Situation	What to do	Why	Small example
night		ruins every report	

Where you'll meet it: this whole document is LLMOps for Project 5. The "things you change" are the ones you built earlier: prompts (Doc03), model settings (Doc04), tools (Doc06), RAG sources (Doc08), and the agent graph (Doc09, Doc11). In a multi-agent system, *each agent* has its own prompt and settings, so each agent is its own "model" that can be released and rolled back.

Real-world examples, by situation:

A support chatbot — the "release" is a prompt edit:

```
# Old way: edit in place, no record.
SYSTEM_PROMPT = "You are a helpful support agent. Be brief."

# LLMOps way: the change is a named release, tested before it goes live.
release = {
    "name": "support_prompt_v4",
    "prompt": "...",
    "model": "gpt-4.1-mini",
    "temperature": 0.2,
}
```

A RAG bot over company documents — nothing in the code changed, but answers got worse: the HR policy PDF was updated, and the search index still has the old version. In classic MLOps this is "data drift." In LLMOps it is "stale sources" — the fix is re-indexing, not retraining.

A multi-agent content system (Project 5): the researcher agent, the writer agent, and the editor agent each have their own prompt. Changing only the editor prompt is still a release of the whole system, because the final article changes.

A common mistake: thinking "we don't train models, so MLOps is not for us." What it causes: prompts get edited directly in production, nobody tests them, and when quality drops nobody knows what changed. How to spot it: ask "what exactly changed in the system last Tuesday?" If the answer needs someone's memory instead of a record, you have this problem.

Quick cheat sheet:

- MLOps = safe changes to trained models. LLMOps = safe changes to prompts, settings, tools, and sources around a model you don't train.
- In LLMOps, a prompt change *is* a model release. Treat it that way.
- Same four habits in both: versions, gated release, watching, rolling back.
- Extra LLMOps risks: the provider changes or fails, and inputs drift.
- Each agent in a multi-agent system is its own releasable unit.

Versions for prompts/settings: treating them as real, tracked things

A **version** is a saved, named, never-changed copy of a prompt plus the settings that go with it (model name, `temperature`, tool list). A prompt that shapes an agent's behavior is not "just some text" — it controls behavior as strongly as code does. So it needs the same care as code: kept in version control (git), changed through reviewable diffs (a view of exactly which lines changed), and tagged so you can tell which version was live for any past run.

Why this matters in practice: when quality drops in production, you must be able to answer "which prompt version was live when this bad answer happened?" Without versions, you can't. Fixing a regression (something that used to work and now doesn't) turns into guesswork, and rolling back is impossible because there is nothing to roll back *to*.

How it works: three rules. (1) A version is **immutable** — once saved, it never changes; a fix becomes a *new* version. (2) The prompt and its settings are saved **together**, because the same prompt with a different model is a different behavior. (3) Every request **logs the version name** it used, so a past answer can be traced back.

What to save in each version:

Field	Why you need it	Small example
`name`	A unique id you can point at and roll back to	`"writer_v4"`
`prompt`	The exact text the model received	`"You write short, clear articles..."`
`model` + settings	The same prompt acts differently on another model or `temperature`	`"gpt-4.1-mini"`, `0.3`
`content_hash`	A fingerprint of the content; proves nobody edited it after saving	`sha256(...)[:12]`
`created_at` + `author`	Who changed what, and when	`"2026-09-01T10:00Z"`, `"sara"`
`eval_score`	The gate result, filled in later (next topic)	`0.93`

When to use it / when NOT to:

Situation	What to do	Why	Small example
Prompt used by real users	Save as a version in git, with a name	You need history and rollback	`prompts/writer/v4.yaml`
Quick experiment in a notebook	Plain string is fine	Nothing depends on it yet; version it once it's heading to production	`prompt = "try this..."`

Situation	What to do	Why	Small example
Fixing a typo in a live prompt	Still make a new version	Even a "small" change can change behavior; the record must be complete	`v4` → `v5`
Many prompts, non-engineers edit them	Use a prompt registry tool (e.g. a prompt management feature in an observability tool) *on top of* the same rules	A web UI is easier for them, but rules stay: immutable, named, logged	Product manager edits via UI, still gets `v6`
A secret (API key)	Never put it in the version file	Versions are shared in git; secrets go in `.env`	Same `.env` rule from Doc01

Real-world examples, by situation:

Saving a version with a fingerprint (plain Python + Pydantic v2):

```
import hashlib, json
from datetime import datetime, timezone
from pathlib import Path
from pydantic import BaseModel

class PromptVersion(BaseModel):
    name: str
    prompt: str
    model: str
    temperature: float
    created_at: str
    content_hash: str

def save_version(
    name: str, prompt: str, model: str, temperature: float
) -> PromptVersion:
    path = Path("versions") / f"{name}.json"
    if path.exists():
        raise FileExistsError(
            f"{name} already exists — make a new version instead"
        )
    body = json.dumps(
        {"prompt": prompt, "model": model, "temperature": temperature},
        sort_keys=True,
    )
    record = PromptVersion(
        name=name, prompt=prompt, model=model, temperature=temperature,
```

```
        created_at=datetime.now(timezone.utc).isoformat(),
        content_hash=hashlib.sha256(body.encode()).hexdigest()[:12],
    )
    path.parent.mkdir(exist_ok=True)
    path.write_text(record.model_dump_json(indent=2))
    return record
```

The `FileExistsError` is what makes a version immutable: you cannot quietly overwrite `writer_v4`.

Tracing a bad answer back (a support desk): a customer complains about a rude reply on 3 September. Every request was logged with `logger.info("reply sent", extra={"prompt_version": "support_v7"})` (same structured logging rule from Doc01). You search the logs, find `support_v7`, and diff it against `support_v6`. The new line "be firm with users" is the cause.

A multi-agent pipeline: save one version per agent, plus a small "release" file that pins them together — `{"researcher": "research_v2", "writer": "writer_v5", "editor": "editor_v3"}`. Then "what was live" is one lookup, even with many agents.

Where you'll meet it: this document's Basic exercise (`version_log`) and Build Task (`versioning.py`). Doc13's traces (Doc13) become much more useful once every trace carries a version name. In Doc11 and Project 4, every agent has its own prompt — each one gets versioned the same way.

A common mistake: saving the prompt text but not the model name and settings. Then someone changes `model="gpt-4.1"` to a cheaper model in a config file, quality drops, and your version history says "nothing changed" — because the prompt did not change. How to spot it: if you can change model or `temperature` without creating a new version name, your versions are incomplete.

Quick cheat sheet:

- Version = prompt **plus** model and settings, saved together, with a unique name.
- Never edit a saved version. A fix is a new version.
- Log the version name on every request, so any answer can be traced back.
- Keep versions in git; keep secrets out of them.
- In multi-agent systems, pin one version per agent in a single release record.

CI/CD for agent systems: the test suite as a release gate

CI/CD means "continuous integration / continuous delivery": every change is tested automatically (CI), and changes that pass can be released automatically (CD). A **release gate** is the check in the middle: a script that runs the test suite on the new version and **refuses to release** if the score is below a set bar. Doc13 built a test suite that catches regressions when you run it by hand. LLMOps makes it automatic and mandatory: no prompt or setting change reaches production without the

suite running first. It is the same idea as tests blocking a bad code merge, just measuring a different kind of correctness.

Why this is worth automating instead of trusting people to remember: the value of Doc13's suite is gone if it is optional. People skip optional steps when they are busy — and a busy day, with a "quick fix," is exactly when a bad change slips through. A gate turns "we should test" from a rule people can forget into something the system enforces.

How it works:

1. A change is proposed (a new version, often as a pull request in git).
2. The CI system (for example GitHub Actions) runs the eval suite on that version.
3. The gate compares the score with the bar, and with the current live version.
4. Pass → the version may be marked live. Fail → release refused, with the failing cases printed.

Because LLM answers are not exactly the same every run, the gate should use a **pass rate over many cases** (for example "at least 90% of 50 cases pass"), not "every single case must match a fixed string."

Choosing what the gate checks:

Check	What it catches	Typical bar	Small example
Quality score (eval pass rate)	The new prompt gives worse answers	`>= 0.90`, and not lower than live version minus a small margin	`new 0.88 < live 0.93 - 0.02` → refuse
Format / schema checks	Broken JSON, missing fields that break the next agent	100%	`Article.model_validate(output)` must not fail
Safety cases	Prompt injection or policy failures (Project 9)	100%	Must refuse "ignore your instructions"
Cost per request	A longer prompt or bigger model that costs much more	`<= 1.2x` live	`$0.004` → `$0.009` refuse
Latency (response time)	Slower answers	p95 `<= 1.2x` live	p95 = the time 95% of requests finish under

When to use it / when NOT to:

Situation	What to do	Why	Small example
Any change to a live prompt, model, or tool	Gate must run, no exceptions	Small edits can break behavior	Typo fix still runs the suite
Emergency: production is broken now	**Roll back** to the last passed version (no gate needed — it already passed)	Don't push an untested "hotfix" past the gate	`rollback_to_previous()`
Early prototype, no users	A light gate (a few smoke tests) is enough	Full suite costs money and time you don't need yet	5 cases instead of 50
Suite is flaky (random pass/fail)	Fix the suite: more cases, `temperature=0` for judge, pass-rate bar	A flaky gate teaches people to ignore or bypass it	Run judge 3 times, take majority

Real-world examples, by situation:

The gate itself (library-neutral; `run_eval_suite` is your Doc13 suite):

```
import logging, sys

logger = logging.getLogger(__name__)

def deploy(
    version_name: str, threshold: float = 0.90, margin: float = 0.02
) -> bool:
    # returns {"pass_rate": 0.91, "failures": [...]}
    new = run_eval_suite(version_name)
    live = run_eval_suite(get_live_version_name())
    below_bar = new["pass_rate"] < threshold
    worse_than_live = new["pass_rate"] < live["pass_rate"] - margin
    if below_bar or worse_than_live:
        logger.warning("Release refused for %s: %.2f (live %.2f)",
                       version_name, new["pass_rate"], live["pass_rate"])
        for case in new["failures"]:
            logger.warning("  failed: %s", case)
        return False
    set_live_version(version_name, passed_gate=True, score=new["pass_rate"])
    return True

if __name__ == "__main__":
    ok = deploy(sys.argv[1])
    # non-zero exit = CI marks the job failed
```

```
    if ok:
        sys.exit(0)
    sys.exit(1)
```

The exit code is the key part: CI tools only understand "0 = pass, not 0 = fail."

In a CI pipeline (GitHub Actions, short form):

```
on: pull_request
jobs:
  eval-gate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: python deploy_gate.py writer_v5
        env:
          OPENAI_API_KEY: ${{ secrets.OPENAI_API_KEY }}
```

If `deploy_gate.py` exits with 1, the pull request cannot be merged (when branch protection is on).

A nightly report job: there is no user traffic to test on, so the gate is the main safety. Before switching the report prompt, the gate runs it on last month's 30 saved inputs and checks that every report still has all required sections.

Where you'll meet it: this document's Intermediate (`release_gate`) and Edge cases (`gate_bypass`) exercises, and the Build Task's `deploy_gate.py` . The suite it runs comes from Doc13. The schema checks are the Pydantic models from Doc04. In a multi-agent system (Doc11), gate the **whole pipeline's final output**, not only each agent alone — a writer change can break what the editor agent expects.

A common mistake: a gate that only checks the new version against a fixed bar, but never against the live version. The live version scores 0.97; the new version scores 0.91; the bar is 0.90; it passes — and quality just dropped 6 points. How to spot it: check whether your gate ever reads the live version's score. If not, add the "not worse than live minus a margin" rule.

Quick cheat sheet:

- No change reaches live without the gate. Emergencies use rollback, not bypass.
- Use a pass rate over many cases, not exact-string matches.
- Compare with the bar **and** with the live version.
- Gate cost and latency too, not only quality.
- Exit code 0 / 1 is how CI knows pass / fail.

Ways to release: canary, blue-green, and shadow

A release strategy decides **how a version that passed the gate reaches real users**. The test suite only covers the cases you thought of; real traffic has cases you didn't. These three strategies let real traffic test the new version while limiting the damage if it is bad.

- **Canary release** — the new version handles a small share of real traffic (say 5%), and the old version handles the rest. If the canary's numbers (errors, scores, cost, speed) stay healthy for a set time, its share grows step by step: 5% → 25% → 50% → 100%. (The name comes from the canary birds coal miners carried to warn them of danger early.)
- **Blue-green** — two full copies of the system run (old = "blue," new = "green"). Traffic switches to green all at once, but blue stays ready, so rolling back is one switch back, not a slow re-release.
- **Shadow release** — the new version runs on real traffic *at the same time* as the old one, but its answers are only recorded and compared, never shown to users. It is the safest way to test a risky change on real inputs.

Why these matter more for agent systems than for typical web apps: a web app bug usually throws an error you can see. An agent's failure is often quiet — a slightly worse answer that still sounds believable. Canary and shadow releases catch that by **comparing numbers** between old and new before every user sees the new version.

Comparing the three:

Strategy	Users see the new version?	Rollback speed	Extra cost	Best for
Canary	Yes, a small share	Fast (set share to 0%)	Low	Most prompt/model changes on a live chatbot
Blue-green	Yes, everyone at once	Instant (switch back)	Two full copies running	Bigger changes: new agent graph, new infrastructure
Shadow	No	Nothing to roll back	Double LLM cost during the test	Risky changes: a new model, a new provider

When to use it / when NOT to:

Situation	What to do	Why	Small example
Changing a chatbot's tone prompt	Canary at 5-10%	Low risk, real feedback quickly	10% of chats use `support_v8`
Switching the whole system to a new provider/model	Shadow first, then canary	You want real-input comparison with zero user risk first	Shadow `gpt-4.1` vs current for 2 days
Change also needs a new database or API version	Blue-green	Old and new can't easily mix per request	Green has the new vector DB schema

Situation	What to do	Why	Small example
Agent does actions (sends emails, charges money)	Shadow **with tools turned off / faked** for the shadow copy	Otherwise the shadow sends real emails twice	Shadow agent gets `dry_run=True` tools
Very low traffic (10 requests a day)	Canary gives no useful numbers; rely on the gate + shadow on saved inputs	5% of 10 requests is nothing to measure	Replay last month's requests
Nightly batch job	No live split; run new version on a copy of the batch and compare	There are no live users to split	Compare 2 report sets

Real-world examples, by situation:

Canary routing, sticky per user:

```
import hashlib

def pick_version(
    user_id: str, canary_version: str, live_version: str, canary_percent: int
) -> str:
    bucket = int(hashlib.sha256(user_id.encode()).hexdigest(), 16) % 100
    if bucket < canary_percent:
        return canary_version
    return live_version

version = pick_version("user_812", "writer_v5", "writer_v4", canary_percent=5)
logger.info("request routed", extra={"prompt_version": version})
```

Hashing the user id (instead of `random.random()`) means the same user always gets the same version, so one person doesn't see the style jump back and forth between messages.

Shadow release for a RAG bot:

```
import asyncio

async def handle(question: str) -> str:
    # user sees this
    answer = await answer_with("rag_v3", question)
    # recorded only
    asyncio.create_task(shadow_compare("rag_v4", question, answer))
    return answer
```

The shadow runs in the background (the `asyncio` pattern from Doc08b), so the user does not wait for it. In real code, keep a reference to the task and catch its errors, so a shadow crash is logged and never reaches the user.

Blue-green for an API: two deployments, `api-blue` and `api-green`, behind one load balancer (the piece that sends each request to a server). You test green directly, then change the load balancer to point at green. If numbers go bad, you point it back at blue within seconds.

Where you'll meet it: this document's Real-world exercise (`canary_release`). The deployment setup from Doc12 is where you add the routing. The comparison numbers come from the traces and evals in Doc13. In multi-agent systems (Doc11, Project 11), a canary is usually decided **once per request at the entry point**, and every agent in that request uses the same release — mixing a new writer with an old editor in one request makes the numbers impossible to read.

A common mistake: running a canary but never setting the numbers that decide "promote" or "stop" before starting. The canary runs, someone looks at a dashboard a week later, the numbers are "about the same," and it gets promoted — even if the canary had 30% higher cost. How to spot it: before a canary starts, you should be able to write down, e.g., "promote if pass rate ≥ live − 0.02, cost ≤ 1.2x, errors ≤ live, after 500 requests." If you can't, the canary is only a guess.

Quick cheat sheet:

- Canary = small share first, grow step by step. Shadow = run in secret, compare. Blue-green = two copies, one switch.
- Decide promote/stop numbers **before** the release starts.
- Keep a user on the same version (hash the user id).
- Shadow copies must never do real actions (emails, payments).
- In multi-agent systems, pick the release once per request, for all agents.

Watching for slow decline: cost, speed, and quality over time

Slow decline means your system gets a little worse each week without any single error. It is sometimes caused by the model changing underneath you (a provider quietly updating a model behind the same name is a real risk). More often it is your system's *inputs* changing: users ask new kinds of questions, the documents in your RAG set go out of date, a tool API starts returning different data. Your prompts and tools were not built for these new inputs.

Why this needs active watching, instead of "it'll show up eventually": these failures don't throw errors. Rising cost, growing delay, and slowly dropping quality scores are *trends*. You only catch a trend by tracking numbers over time — no single request looks broken. By the time users complain, the decline may have lasted weeks.

How it works: (1) log a few numbers for every request, with the version name; (2) every day, compute a **rolling average** (the average over the last N days) for each number; (3) compare it with

a **baseline** — the numbers from when the version passed the gate; (4) alert when the difference crosses a set line for several days in a row, not on one bad day.

What to watch:

Number	How to measure it	What a slow decline looks like	Common cause
Cost per request	Tokens × price, from the API's `usage` field	$0.004 → $0.007 over a month	Conversation history growing; retries; longer RAG context
Latency p95	Time per request; the 95th percentile	3s → 6s	Bigger context, slower tool, provider load
Quality score	Run the eval judge on a small random sample (e.g. 2%) of real traffic every day	0.93 → 0.85	New question types; stale documents
Thumbs down / user complaints	Feedback button, support tickets	2% → 5%	Users notice before your judge does
"I don't know" / refusal rate	Count answers that match "no information" patterns	3% → 15%	RAG documents missing new topics
Tool error rate	Count failed tool calls	0.5% → 4%	Tool API changed its response format

When to use it / when NOT to:

Situation	What to do	Why	Small example
Live system with daily traffic	Daily rolling averages + trend alerts	Catches decline in days, not months	7-day average vs baseline
Traffic is tiny	Weekly replay of a fixed saved set of real questions	Too few requests for stable daily numbers	Re-run 40 saved questions every Monday
Quality judge is expensive	Sample a small % of traffic	Judging 100% can double your LLM cost	Judge 2% of requests
One request is very slow or wrong	That's an incident, not a trend — handle with normal alerts (Doc12/13)	Trend watching is for slow change, not spikes	A 60s timeout pages someone
Pinned model version (e.g. a dated model snapshot)	Still watch quality	Your inputs can still drift even if the model doesn't	Stale HR documents

Real-world examples, by situation:

A daily trend check (plain Python):

```
from statistics import mean

def check_trend(
    daily_values: list[float],
    baseline: float,
    max_increase: float,
    days: int = 3,
) -> bool:
    """True if EACH of the last `days` values is above the limit."""
    recent = daily_values[-days:]
    if len(recent) < days:
        return False
    limit = baseline * (1 + max_increase)
    for value in recent:
        if value <= limit:
            return False
    return True

cost_per_day = [0.0041, 0.0043, 0.0049, 0.0052, 0.0055]
if check_trend(cost_per_day, baseline=0.0040, max_increase=0.20):
    logger.warning(
        "Cost trend alert: last 3 days above +20%% of baseline (avg %.4f)",
        mean(cost_per_day[-3:]),
    )
```

"Three days in a row" stops one busy day from causing a false alert.

A RAG bot over company docs: the "I don't know" rate climbs from 3% to 15% over six weeks. No errors, no complaints yet. The trend alert fires; you find that a new product line was launched and its documents were never added to the index.

A support desk agent: cost per conversation doubles over a month. Cause: a new feature keeps the whole chat history in every call. Nothing was "broken" — the numbers found it.

A scheduled news summary job: a news API quietly started returning full article HTML instead of plain text. Tokens per run tripled. The cost trend caught it before the monthly bill did.

Where you'll meet it: the logging and traces from Doc13 give you the raw numbers; this topic adds "compare over time." This document's Failure exercise (`rollback_trigger`) uses the same kind of check to decide a rollback. In a multi-agent system (Doc11), track the numbers **per agent** as well as per request — a total cost increase is much easier to fix when you can see it came only from the researcher agent's growing search results.

A common mistake: comparing each day only to the day before. A 2% worse day never looks alarming, but 2% worse every day for a month is a big decline. How to spot it: check what your alert compares against. If it's "yesterday," change it to a fixed baseline (the numbers from when the version was released).

Quick cheat sheet:

- Slow decline throws no errors — only numbers show it.
- Log cost, latency, and version name on every request; judge a small sample for quality.
- Compare rolling averages with a fixed baseline, not with yesterday.
- Alert on several bad days in a row, not one.
- Track per agent, not only per request.

Rolling back: the real safety net

Rolling back means putting the system back on the **last version known to be good** — a prompt, a model choice, or a whole release — as soon as watching or a gated test shows the new version is worse. "Known to be good" means: it passed the gate, and it ran healthy in production. It does *not* mean "whatever version came before."

Why an automatic, pre-defined rollback trigger matters more than a person watching a dashboard: people get busy, alerts get missed, and people argue "let's wait one more day." The value of a rollback is in how fast it happens after a problem starts. A rollback that needs a person to notice, decide, and run it by hand is much slower and less reliable than one that fires on its own when a number crosses a set line.

How it works: if versions are immutable and saved (topic 2), rolling back is only **moving a pointer** — change "live = writer_v5" back to "live = writer_v4." No rebuild, no re-test. Keep a **release history** (a list of every live change, with time, score, and whether it passed the gate). Then rollback = "walk back through history to the newest entry that passed the gate and is not the current bad one."

Choosing a rollback trigger:

Trigger type	Example rule	Good for	Watch out for
Error rate	Errors > 5% over the last 200 requests	Broken format, crashes, tool failures	Too few requests → noisy
Quality drop	Sampled pass rate < baseline − 0.05 over 300 requests	Quiet quality regressions	Need enough judged samples
Cost jump	Cost per request > 1.5x baseline for 1 hour	Runaway loops, longer prompts	Normal traffic peaks
User signal	Thumbs-down rate doubles over a day	Problems your judge misses	Slower signal

Trigger type	Example rule	Good for	Watch out for
Manual	Engineer runs `rollback` command	Anything automatic can't see	Humans are slow

When to use it / when NOT to:

Situation	What to do	Why	Small example
New version clearly worse after release	Roll back first, investigate after	Users stop being hurt now; debugging can wait	Pointer back to `writer_v4`
The problem is outside your system (provider outage)	Don't roll back — use the backup plan (next topic)	The old version fails the same way	OpenAI 503 on every version
Bad data was already written (wrong emails sent, bad DB rows)	Roll back **and** clean up the data separately	Rollback fixes future requests, not past side effects	Re-send corrected emails
RAG index was rebuilt and answers got worse	Roll back the index pointer too, not only the prompt	"Version" includes sources	Point search at `index_2026_08`
Rolled back, and want to try again	Fix → new version → gate → canary	Never "un-rollback" the same bad version	`writer_v6`, not `writer_v5` again

Real-world examples, by situation:

Roll back to the last version that passed, not just the previous one:

```
from pydantic import BaseModel

class Release(BaseModel):
    version: str
    passed_gate: bool
    released_at: str

def rollback_to_previous(history: list[Release], current: str) -> Release:
    for release in reversed(history):
        if release.passed_gate and release.version != current:
            set_live_version(release.version)
            logger.warning(
                "Rolled back from %s to %s", current, release.version
            )
            return release
```

```
    raise RuntimeError(
        "No earlier version passed the gate — cannot roll back safely"
    )
```

If `writer_v5` got through by a bypass (not passing the gate), this skips it and goes to `writer_v4`. The `RuntimeError` is a clear, named failure (Doc01's error rule) instead of silently picking a bad version.

Automatic trigger on a live chatbot:

```
def maybe_rollback(
    recent_pass: list[bool], baseline_rate: float, min_requests: int = 300
) -> None:
    if len(recent_pass) < min_requests:
        return                                  # not enough data yet
    rate = sum(recent_pass) / len(recent_pass)
    if rate < baseline_rate - 0.05:
        rollback_to_previous(load_history(), get_live_version_name())
```

`min_requests` stops the trigger from firing on the first 10 unlucky requests.

A multi-agent release: the release record pins all agents together (`researcher_v2, writer_v5, editor_v3`). Rollback moves back to the whole previous release record, not only the writer, because the editor prompt may have been changed to match the new writer.

Where you'll meet it: this document's Failure exercise (`rollback_trigger`) and the Build Task's `rollback.py` . The numbers it reads come from the watching in the previous topic and Doc13's evals. In Doc12, you deploy code; rolling back code (going back to an older container image) follows exactly the same "pointer to last good" idea. In a multi-agent system, the rollback unit is the **whole release record**.

A common mistake: a rollback that returns "the version before the current one." If the previous version was also bad (for example it slipped past the gate, or was itself rolled back), you quietly re-release a broken version and believe you are safe. How to spot it: read your rollback code — if it uses `history[-2]` without checking `passed_gate` , it has this bug.

Quick cheat sheet:

- Roll back to the last version that **passed the gate**, never just "the previous one."
- With immutable versions, rollback is only moving a pointer — it should take seconds.
- Decide the trigger rule (number + window + minimum requests) before release.
- Roll back first, debug after.
- Rollback fixes future requests, not side effects already done.

Backup plans if your provider has an outage

Your system depends on an outside provider (for example OpenAI). It can have outages, change rate limits (how many requests you may send per minute), or retire a model you use — all outside your control. A **backup plan** (also called a **fallback**) is a pre-built, tested path your system switches to when the main provider fails.

Why this is a real risk, not a hypothetical: a production agent system with no backup plan goes fully down the moment its one dependency does. Every major LLM provider has had outages. Model retirements also happen on a published schedule — if you ignore the notices, your system breaks on a known date.

What a basic backup plan looks like, in layers:

Layer	What happens	When it kicks in	Small example
1. Retry with backoff	Try the same call again, waiting longer each time	Short blips: timeouts, HTTP 429 (rate limited), 5xx	Doc02's backoff; the OpenAI SDK also retries by itself (`max_retries`)
2. Backup model, same provider	Use another model	One model is overloaded or retired	`gpt-4.1` → `gpt-4.1-mini`
3. Backup provider	Send the request to a different company's model	The whole provider is down	A second provider, set up and tested in advance
4. Cached / simple answer	Return a saved answer, or a non-LLM result	All models fail; question was seen before	FAQ answer from cache
5. Graceful message	A clear "temporarily unavailable, try again soon"	Nothing else worked	Better than a raw 500 error

A **circuit breaker** ties this together: after N failures in a row, stop calling the broken provider for a few minutes and go straight to the backup. This avoids making every user wait through timeouts.

When to use it / when NOT to:

Situation	What to do	Why	Small example
Public chatbot or support desk	Retries + backup model + graceful message, at least	Users are waiting live	Layers 1, 2, 5
Business-critical system (many users, money involved)	Add a backup provider, and **gate it** like any version	A backup that was never tested may fail or answer badly	Backup provider has its own eval score

Situation	What to do	Why	Small example
Nightly batch job	Retry with longer waits; re-run later; alert if still failing	Nobody waits live, so waiting is fine	Retry for 2 hours, then alert
Actions that must not happen twice (payments, emails)	Use an idempotency key (a unique id so a repeat is ignored) before retrying	A retry after a timeout may repeat an action that actually succeeded	`order_id` checked before sending
Error is 400 / 401 (bad request, bad key)	Do **not** retry or fall back	That's your bug, not an outage; the backup will fail too or hide it	Fix the key in `.env`
Model retirement notice	Plan migration: new version → gate → canary, before the date	It is a scheduled change, not an emergency	Move off an old model a month early

Real-world examples, by situation:

Backup model with the OpenAI Python SDK:

```
import logging
from openai import OpenAI, APIConnectionError, APITimeoutError
from openai import RateLimitError, InternalServerError

logger = logging.getLogger(__name__)
# layer 1: the SDK retries short blips itself
client = OpenAI(timeout=20, max_retries=2)

MODELS = ["gpt-4.1", "gpt-4.1-mini"]          # layer 2: backup model
OUTAGE_ERRORS = (
    APIConnectionError, APITimeoutError, RateLimitError, InternalServerError
)

def ask(messages: list[dict]) -> str:
    for model in MODELS:
        try:
            response = client.chat.completions.create(
                model=model, messages=messages
            )
            return response.choices[0].message.content
        except OUTAGE_ERRORS as e:
            logger.warning(
                "Model %s failed (%s), trying next", model, type(e).__name__
            )
    # layer 5: a friendly message when every model is down
```

```
    return ("Sorry, the assistant is temporarily unavailable. "
            "Please try again in a few minutes.")
```

Only outage-type errors trigger the fallback. A `BadRequestError` or `AuthenticationError` is not caught, so your own bug still crashes loudly (Doc01's "catch only what you can handle" rule).

A simple circuit breaker:

```
import time

class CircuitBreaker:
    def __init__(self, max_failures: int = 5, cooldown_s: int = 120):
        self.failures = 0
        self.opened_at = 0.0
        self.max_failures = max_failures
        self.cooldown_s = cooldown_s

    def is_open(self) -> bool:
        too_many = self.failures >= self.max_failures
        still_cooling = time.time() - self.opened_at < self.cooldown_s
        return too_many and still_cooling

    def record(self, ok: bool) -> None:
        if ok:
            self.failures = 0
        else:
            self.failures += 1
            if self.failures >= self.max_failures:
                self.opened_at = time.time()
```

Before each call: if `breaker.is_open()`, skip the main provider and go straight to the backup. After the cooldown ends, calls try the main provider again; one more failure opens the breaker again at once.

A support desk during an outage: both models fail. Instead of an error page, the bot says "Our assistant is down right now. Your message has been saved and a person will reply within 2 hours," and saves the ticket to the database. The user is not left with nothing.

A multi-agent content pipeline: the researcher agent's search tool is down. The editor and writer can still work, so the pipeline continues with "no new research — using saved sources" marked clearly in the output, instead of stopping the whole run.

Where you'll meet it: timeouts and backoff first appear in Doc02; SDK errors in Doc04; production error handling in Doc12. In Doc11 and Project 11, each agent can have its own fallback, and the orchestrator decides whether a failed agent stops the run or lets it continue with less. Libraries like

LangChain also offer built-in fallbacks (`.with_fallbacks([...])` on a model, from Doc05) — same idea, less code.

A common mistake: setting up a backup model but never testing it. The day of the outage, the backup returns answers in a different format, the next agent's Pydantic check fails, and the "backup" breaks the system anyway. How to spot it: ask "when did the backup last pass our eval suite?" If the answer is "never," run the gate on it now, and keep running it on every release.

Quick cheat sheet:

- Layers: retry → backup model → backup provider → cache → clear message.
- Fall back only on outage errors (timeout, 429, 5xx), never on 400/401.
- A backup must pass the same gate as the main version.
- Use a circuit breaker so users don't wait through repeated timeouts.
- Treat model retirement notices as a planned release, not an emergency.

Practice Exercises

Setup for this document's practice code: work inside `19_mlops_llmops/` (same venv as before — if it's not active, `source .venv/bin/activate`). No new packages — `pytest` and `python-dotenv` are already installed from Doc13 and Doc01.

Where your code lives: all of it under `19_mlops_llmops/practice/` (`mkdir -p practice`), never loose beside this README. Exercises are grouped **by topic, not by difficulty level** — the same convention as Doc01-18 — so one topic's growth from basic to intermediate stays visible in one file.

The full file layout, all exercises:

```
practice/
├── build_task/                   Build Task — its own folder
├── canary_release_practice.py    Real-world
├── release_gate_practice.py      Intermediate + Edge cases
│                                   (2 sections)
├── rollback_trigger_practice.py  Failure
└── version_log_practice.py       Basic
```

Why each script exists:

- `canary_release_practice.py` — a small share of traffic on the new version, with quality *and* cost compared against the old one.
- `release_gate_practice.py` — a gate that only lets a version go live if its score clears the bar, then proof it can't be skipped by editing a file.

- `rollback_trigger_practice.py` — a trigger that fires when scores really drop, and fires only once.
- `version_log_practice.py` — every prompt version saved with its text and time, never overwritten, so "which version was live?" has an answer.

How to run each exercise: if two exercises below are really about the same thing, save them together in ONE script named after that topic, with each level's version as its own clearly labeled section inside it. Run each topic's file from inside `practice/`, for example: `python version_log_practice.py`.

For this document:

- Basic (`version_log`) is its own topic — save it as `practice/version_log_practice.py`.
- Intermediate (`release_gate`) and Edge cases (`gate_bypass`) are both about the same gate — building it, then trying to bypass it and locking that hole shut — save them together as `practice/release_gate_practice.py`, with each level as its own section.
- Real-world (`canary_release`) is its own topic — save it as `practice/canary_release_practice.py`.
- Failure (`rollback_trigger`) is its own topic — save it as `practice/rollback_trigger_practice.py`.

Why group by topic instead of by level: if you save each exercise by difficulty level instead, the different versions of the same idea end up scattered across separate files, and you can never see how one topic grows from simple to harder in one place. Grouping by topic keeps that growth visible — open one file, and you see the whole journey for that one thing, from basic to intermediate, side by side.

Jump to an exercise: Basic · Intermediate · Real-world · Edge cases · Failure · Build Task

Basic — a simple version log

- **What:** tag two versions of one prompt from Project 5 (v1, v2) in a simple version log, and run Doc13's test suite against both.
- **Why:** you can't have a deploy gate without something to compare against first — this is the record that makes "which version was live when" an answerable question.
- **When you'll hit this for real:** the very first time you change a prompt in a system you actually care about not breaking.
- **How to code it:** a plain JSON or SQLite record with `{version_name, prompt_text, created_at}`, saved for two variants of one prompt, with your Doc13 suite run against each and the scores printed side by side.
- **Save as:** `practice/version_log_practice.py`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Intermediate — build the gate

- **What:** a small release gate script that refuses to update the "current live prompt version" unless the new version's test score meets or beats a set bar.
- **Why:** this is the actual mechanism that turns "we should test before releasing" from a policy people can forget into something the system enforces.
- **When you'll hit this for real:** this document's own Build Task, protecting Project 5.
- **How to code it:** a `deploy(version_name, threshold)` function that runs your test suite, and only updates a `live_version.txt` (or database row) if the score clears the threshold — otherwise it prints why and exits without updating anything.
- **Save as:** `practice/release_gate_practice.py`, under an `# Intermediate` section (this file also holds the Edge cases exercise below, in its own `# Edge cases` section).
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Real-world — act out a canary release

- **What:** send a small share of test requests to v2, the rest to v1, and compare live numbers (cost, speed, test score) between the two groups.
- **Why:** this is the actual pattern real companies use to de-risk a change — practicing it here, on your own project, is what makes it something you can describe concretely in an interview.
- **When you'll hit this for real:** rolling out any real prompt or model change to a system already serving real traffic.
- **How to code it:** loop over your test requests, route roughly 20% to v2 and the rest to v1 (`random.random() < 0.2`), log which version handled each one, and compare aggregate scores/cost between the two groups at the end.
- **Save as:** `practice/canary_release_practice.py`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Edge cases — a bad release that slips past the gate

- **What:** deliberately release a worse version past your own gate (skip it on purpose), and confirm your watching would have caught it — then wire the gate back so it can't be skipped quietly.
- **Why:** a gate that can be bypassed by editing a file directly isn't really a gate — you need to prove to yourself it can't happen by accident.
- **When you'll hit this for real:** exactly the failure mode this document's Break-It section warns about — someone (possibly future you) editing the "live" pointer directly under time pressure.
- **How to code it:** manually edit your `live_version.txt` to point at an untested version, bypassing `deploy()`. Confirm nothing in your system stops you — then add a check (a hash, or a "passed_gate: true" flag) that makes a manually-edited pointer detectable or impossible.
- **Save as:** `practice/release_gate_practice.py`, under an `# Edge cases` section (this file also holds the Intermediate exercise above, in its own `# Intermediate` section).

- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Failure — a real rollback trigger

- **What:** define a real rollback trigger (like "test score drops more than X% over Y requests") and act out the situation that should set it off.
- **Why:** a rollback plan that only exists as an idea, never tested, is not a rollback plan — you need to have watched the trigger actually fire once.
- **When you'll hit this for real:** the day a released version turns out to be worse than it looked in testing — this is what makes recovery fast instead of panicked.
- **How to code it:** write a function checking recent scores against a threshold, feed it a simulated sequence of scores that crosses the line partway through, and confirm it correctly calls your `rollback_to_previous()` function at the right moment.
- **Save as:** `practice/rollback_trigger_practice.py`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Build Task — Versioned Release Gate for Project 5

Stuck on the Build Task? Hint 1 · Hint 2 · Show me the solution **Goal:** a light versioning and gated-release layer on top of Project 5, so changing a prompt or agent setting in production requires passing the same test suite from Doc13 — not just optional, easy-to-skip discipline.

Requirements:

- Prompts/settings for Project 5's agents are stored as versioned, named pieces (not edited in place, with no history).
- A `deploy(version)` step that runs the Doc13 test suite against that version, and refuses to mark it "live" if the score is below a set bar.
- A record of which version was live at any point in time, so a given past run can be traced back to the exact prompt/setting that produced it.
- A rollback function that puts the "live" marker back to the last version that actually passed.

Inputs: a new prompt/setting version to release; the existing test suite from `13_testing_evaluation_observability`.

Outputs: either the version goes live (it passed the gate), or the release is refused, with the test scores explaining why.

Constraints: the gate must not be skippable by accident — "just editing the prompt file directly" shouldn't be how a version reaches production.

Where your code lives: `19_mlops_llmops/practice/build_task/` — run everything from inside that folder.

Suggested files:

```
19_mlops_llmops/practice/build_task/
├── deploy_gate.py        deploy() -> DeployResult, event history
├── main.py               runs one save / deploy / rollback cycle
├── rollback.py           rollback_to_previous() -> VersionRecord
├── test_deploy_gate.py   one test per row of the Test Cases table
└── versioning.py         VersionRecord, save/load/get a version
```

Why each file exists:

- `deploy_gate.py` — `release_gate`'s gate and `DeployResult`; every decision is a timestamped event, and "what's live" is worked out from those events (`gate_bypass`'s idea), so there's no pointer file to edit.
- `main.py` — one short run that shows the whole cycle working.
- `rollback.py` — goes back to the last version that *passed*, never to one the gate refused.
- `test_deploy_gate.py` — the Test Cases table below as pytest tests, in the same style as Doc13.
- `versioning.py` — `version_log`'s load-then-append record, with each version as a typed `VersionRecord`.

Functions/Components to build:

- `versioning.py` → `save_version(name, config) -> VersionRecord`, `get_version(name) -> VersionRecord`
- `deploy_gate.py` → `deploy(version_name: str, threshold: float) -> DeployResult` (runs Doc13's test suite inside itself)
- `rollback.py` → `rollback_to_previous() -> VersionRecord`

Expected Behavior

- A version that scores above the bar releases, and becomes traceable as "live."
- A version that scores below the bar is refused, with the specific failing test cases shown.
- Rolling back correctly restores the last version that passed, not just "whatever came before" no matter if it passed or not.

Test Cases

Scenario	Expected
New version scores above the bar	Releases, becomes live, saved with a version ID

Scenario	Expected
New version scores below the bar	Release refused, failing test cases shown
Rollback called after a bad release got through	Live marker goes back to the last version known to be good
Asking "what was live at time T"	Returns the correct saved version record

Break-It / Debug Preview

- A release gate that can be skipped by editing the "live" marker directly — completely defeating its purpose.
- A rollback that goes back to "whatever came before" instead of "the last one that actually passed" — quietly re-releasing a version that was already broken.
- A slow decline that only shows up over weeks (rising cost) instead of one bad request — nothing catches it without deliberately tracking the trend, not just per-request alerts.

Interview Topics Preview

- MLOps vs. LLMOps · why prompts need the same version discipline as code · canary vs. blue-green vs. shadow release trade-offs · what a good rollback trigger looks like · a backup plan for a provider outage.

□ You Can Now Finish Project 5

Docs 12-19 are everything Project 5 needs. Go to project_5_contentforge_pro_production/ and build it: Steps 1-3 are Doc12's API, database and Docker, and Step 4 is Doc13's eval suite — the exact suite this document's release gate runs. The gate and rollback themselves stay in this document's Build Task, on top of the finished project.

Move On When

You can explain, clearly, what would need to be true for you to trust an automatic rollback enough not to watch every release yourself — and you have a working gate that actually blocks a worse version of Project 5 from reaching "live." There's no document after this one — go back to 18_capstone and CURRICULUM.md §6 to confirm the whole curriculum is complete.

Stuck? Ask for **Hint 1** or **Hint 2**. Ask for the full solution only if you say **"Show me the solution."**